GEN AI LEVEL 1 — MASTER CURRICULUM & CAPSTONE

31-Day Mysore Program: Python, Data Science, FastAPI Microservices, LangChain & Enterprise RAG

---

1. Program Overview & Objectives

This 31-day curriculum equips engineering students with full-stack Generative AI competencies. Participants transition from advanced object-oriented Python and scientific computing into building asynchronous, production-grade FastAPI microservices, deploying LangChain LCEL workflows, and building enterprise Retrieval-Augmented Generation (RAG) platforms.

2. Course Structure & Timeline (31 Days Breakdown)

Sl No.	Course Module Name	Duration	Core Pedagogical Deliverables
1	Programming Using Python	4 Days	OOP, generators, decorators, modular packages, custom exceptions,
2	Python for Data Science	4 Days	NumPy array operations, Pandas DataFrames, EDA, token distributio
3	Intro to ML, Deep Learning & GenAI	3 Days	Vector embeddings, cosine similarity, Transformer self-attention, toke
4	API & Microservices Fundamentals	2 Days	RESTful constraints, HTTP methods, status codes, JSON contracts.
5	Advanced Python: FastAPI & Pydantic	4 Days	Pydantic v2 schemas, FastAPI async endpoints, dependency injection
6	Intro to GenAI, LLM, LangChain & Agents	6 Days	Prompt templates, LCEL composition, model wrappers, basic ReAct a
7	Microservices Fundamentals & FastAPI	4 Days	Docker containerization, JWT auth, rate limiting, health probes, Docke
8	Generative AI using LangChain	4 Days	Production RAG, chunking, FAISS/Chroma vector stores, citation grou

3. Production Capstone Project: OmniDocs Intelligence Platform

Repository: https://github.com/ShreyasVavley/genai-l1-omnidocs
OmniDocs is an enterprise-grade document intelligence platform that bridges every topic from the 31-day syllabus. It ingests corporate whitepapers, performs exploratory statistical analysis using Pandas, indexes chunks into a 256-D normalized vector space, and serves queries via an asynchronous FastAPI microservice with LangChain LCEL and an autonomous ReAct agent loop.

Core Architectural Layers:

• **Data & Ingestion:** Recursive paragraph/sentence splitter with 50-character boundary overlap and Pandas corpus analysis.

• **Vector Store & Indexing:** High-speed normalized vector projection with cosine distance matching and keyword boosting.

• **LLM Orchestration:** LangChain Expression Language (LCEL) chain: prompt | llm | output_parser with citations.

• **ReAct Tool Loop:** Dynamic execution of Calculator, Document Search, Summarizer, and System Health tools.

• **Enterprise Security:** Lightweight signed JWT authentication, API Key headers, and timing middleware.

Sample LangChain LCEL Production Chain Code:

```
# Production LangChain LCEL RAG Chain
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = PromptTemplate.from_template(
    "Context: {context}\n\nQuestion: {query}\nAnswer grounded strictly in context:"
)
chain = prompt | llm_model | StrOutputParser()
result = chain.invoke({"context": context_text, "query": user_query})
```

4. Student Hands-on Lab Drills & Weekly Milestones

Week 1 (Days 1-8): Python Foundations & Data Science EDA

Lab 1.1: Build an OOP document reader with custom exceptions and file context managers.
Lab 1.2: Use Pandas to analyze document length distributions, token frequencies, and compute TF-IDF metrics.

Week 2 (Days 9-17): ML Intuition, REST Microservices & FastAPI

Lab 2.1: Implement high-dimensional vector dot products and cosine similarity from scratch using NumPy.
Lab 2.2: Build a FastAPI service with Pydantic v2 validation models, async endpoints, and timing middleware.

Week 3 (Days 18-23): GenAI Prompt Engineering, LangChain & ReAct

Lab 3.1: Construct zero-shot and few-shot prompt templates using LangChain.
Lab 3.2: Implement an autonomous ReAct loop with Calculator and Search tool bindings.

Week 4 (Days 24-31): Enterprise RAG, Dockerization & Capstone Defense

Lab 4.1: Containerize the FastAPI RAG service using Docker and write a multi-container Docker Compose file.
Lab 4.2: Deploy to cloud (Railway/Render) and run test suite with pytest.

5. Top Technical Interview Questions & Evaluation Guide

Q1: Why is asynchronous concurrency in FastAPI preferred for GenAI microservices over traditional Flask/Django?

Answer: GenAI applications spend most of their execution time waiting for external I/O (LLM API calls, vector database queries). FastAPI's ASGI event loop allows thousands of concurrent requests to be handled efficiently on a single process without blocking threads.

Q2: How does chunk overlap prevent context loss in RAG systems?

Answer: When a sentence or semantic thought spans across the arbitrary boundary of two chunks, chunk overlap ensures that both chunks retain sufficient mutual context, preventing semantic truncation during embedding matching.

Q3: What are the primary failure modes of LangChain LCEL chains and how do you monitor them?

Answer: Failure modes include token limit breaches, rate-limiting HTTP 429s, and hallucinations. Mitigation includes fallback chains, retry decorators, and tracing via LangSmith or OpenTelemetry.